

Math Exercise Lab 8

Nguyễn Minh Khang MSSV: 2211452

November 2025

Contents

1 Phương pháp đề xuất

1.1 Tổng quan phương pháp và động cơ lựa chọn mô hình

Trong nghiên cứu này, nhóm đề xuất một phương pháp dự báo ùn tắc giao thông dựa trên **mô hình Transformer encoder-only theo phong cách BERT + Spatial Encoding(Laplacian) + Temporal Encoding**, được thiết kế để học biểu diễn không gian–thời gian trên các *vùng giao thông động* (dynamic traffic zones). Mỗi vùng giao thông được biểu diễn như một đồ thị con (subgraph) bao gồm các tuyến đường có mối liên hệ về không gian, cấu trúc mạng lưới và hành vi giao thông, được khởi tạo từ một sự kiện ùn tắc tại thời điểm nhất định.

Việc lựa chọn kiến trúc Transformer encoder-only + Spatial Encoding(Laplacian) + Temporal Encoding được thúc đẩy bởi những hạn chế cố hữu của các phương pháp học biểu diễn đồ thị dựa trên cơ chế truyền thông tin cục bộ (message passing), chẳng hạn như Graph Convolutional Networks (GCN) hay các biến thể GNN phổ biến khác. Các mô hình này cập nhật biểu diễn nút thông qua việc tổng hợp thông tin từ lân cận k-hop, do đó gặp khó khăn trong việc nắm bắt cấu trúc toàn cục của đồ thị và phân biệt các nút có vai trò cấu trúc khác nhau nhưng có lân cận cục bộ tương tự. Hơn nữa, khi tăng số lớp, các mô hình GNN thường gặp hiện tượng *over-smoothing*, khiến biểu diễn của các nút trở nên khó phân biệt. Ngoài ra, trong bối cảnh dữ liệu chuỗi thời gian dài hạn (long-term temporal dependency) như bài toán giao thông đô thị, các mô hình học tuần tự truyền thống dựa trên Recurrent Neural Networks (RNN) hoặc các biến thể như LSTM/GRU cũng bộc lộ những hạn chế đáng kể. Mặc dù được thiết kế nhằm giảm thiểu hiện tượng suy biến gradient, các mô hình này vẫn gặp khó khăn trong việc học các phụ thuộc thời gian dài khi độ sâu theo thời gian tăng lên, dẫn đến hiện tượng vanishing gradient, làm suy giảm khả năng lan truyền thông tin từ các thời điểm xa trong quá khứ đến đầu ra dự đoán. Tương tự, các phương pháp dựa trên Temporal Convolution với receptive field hữu hạn thường chỉ nắm bắt được các mẫu cục bộ theo trục thời gian, và việc mở rộng phạm vi phụ thuộc đòi hỏi phải tăng độ sâu mạng, từ đó làm gia tăng độ phức tạp và nguy cơ mất ổn định trong huấn luyện.

Những hạn chế trên đã được phân tích và làm rõ trong công trình của Ying et al. [?], trong đó tác giả chỉ ra rằng Transformer có thể đạt hiệu quả cao trong học biểu diễn đồ thị nếu được bổ sung các cơ chế mã hóa cấu trúc phù hợp. Cụ thể, Graphormer cho thấy rằng việc đưa thông tin cấu trúc của đồ thị trực tiếp vào cơ chế self-attention cho phép mô hình vượt qua giới hạn biểu diễn của các kiến trúc GNN truyền thống, đồng thời cung cấp trường nhìn toàn cục (global receptive field) cho mỗi nút trong đồ thị.

1.2 Mỗi tương quan giữa các tuyến trong giao thông đô thị

Dựa trên những phân tích này, nhóm xây dựng mô hình Transformer encoder-only để học biểu diễn không gian–thời gian cho từng vùng giao thông. Tuy nhiên, khác với các bài toán đồ thị tĩnh, bài toán giao thông đô thị đòi hỏi mô hình phải nắm bắt được **tương tác động và có độ trễ theo thời gian giữa các tuyến đường**, phản ánh cơ chế lan truyền của ùn tắc trong mạng lưới giao thông.

Để giải quyết yêu cầu này, nhóm sẽ khai thác các mối tương quan chéo có xét đến độ trễ thời gian giữa các tuyến đường, dựa trên khuôn khổ mạng tương quan giao thông được đề xuất bởi Guo et al. [?]. Cách tiếp cận này cho phép định lượng mức độ ảnh hưởng giữa các tuyến đường và xác định các tuyến có vai trò quan trọng trong quá trình lan truyền ùn tắc.

Cụ thể, với mỗi cặp tuyến đường i và j , từ chuỗi thời gian vận tốc đã được chuẩn hóa $S_i(t)$ và $S_j(t)$, hàm tương quan chéo có độ trễ được định nghĩa như sau:

$$X_{i,j}(\tau) = \frac{\sum_{t=1}^{L-\tau} (S_i(t) - \bar{S}_i) (S_j(t + \tau) - \bar{S}_j)}{\sqrt{\sum_{t=1}^{L-\tau} (S_i(t) - \bar{S}_i)^2 \sum_{t=1}^{L-\tau} (S_j(t + \tau) - \bar{S}_j)^2}}, \quad (1)$$

trong đó $\tau \in [-\tau_{\max}, \tau_{\max}]$ là độ trễ thời gian, L là độ dài chuỗi thời gian, và $\bar{S}_i, \bar{S}_j$ lần lượt là giá trị trung bình của các chuỗi tương ứng. Với $\tau < 0$, ta có $X_{i,j}(\tau) = X_{j,i}(-\tau)$.

Từ hàm tương quan chéo này, Guo et al. [?] đề xuất trọng số liên kết dương (positive link weight) giữa hai tuyến đường i và j được xác định bởi:

$$W_{i,j}^{\text{pos}} = \frac{\max_{\tau} X_{i,j}(\tau) - \text{mean}_{\tau} (X_{i,j}(\tau))}{\text{std}_{\tau} (X_{i,j}(\tau))}, \quad (2)$$

trong đó $\max_{\tau}$, mean_{τ} và std_{τ} lần lượt là giá trị lớn nhất, trung bình và độ lệch chuẩn của hàm tương quan chéo trên miền τ . Đồng thời, độ trễ tương ứng với ảnh hưởng mạnh nhất được xác định là:

$$\tau_{i,j}^{\text{pos}} = \arg \max_{\tau} X_{i,j}(\tau). \quad (3)$$

Trọng số $W_{i,j}^{\text{pos}}$ phản ánh mức độ nổi trội của đỉnh tương quan so với nền nhiễu, giúp lọc bỏ các tương quan yếu hoặc ngẫu nhiên, trong khi $\tau_{i,j}^{\text{pos}}$ cung cấp thông tin về hướng và độ trễ của ảnh hưởng giữa các tuyến đường. Việc sử dụng các đại lượng này cho phép nhóm xác định các liên kết giao thông mang ý nghĩa chức năng, phù hợp để mô hình hóa quá trình lan truyền ùn tắc trong không gian và thời gian.

Trên cơ sở các liên kết tương quan có ý nghĩa thống kê và ràng buộc không gian, nhóm xây dựng các vùng giao thông chức năng và xác định tập các tuyến mục tiêu cần dự đoán. Các vùng này sau đó được đưa vào mô hình Transformer encoder-only với các cơ chế mã hóa không gian–thời gian, cho phép học biểu diễn giàu ngữ cảnh và nâng cao khả năng dự báo ùn tắc giao thông trong môi trường đô thị phức tạp.

1.3 Xây dựng vùng giao thông động

Như đã trình bày ở Mục ??, vùng giao thông trong nghiên cứu này không được xem là một khu vực không gian cố định, mà là một *vùng chức năng động* (functional dynamic zone) được khởi tạo từ các sự kiện ùn tắc và mở rộng dựa trên mối liên hệ không gian–thời gian giữa các tuyến đường. Quá trình xây dựng vùng giao thông động bao gồm hai giai đoạn chính: (i) phát hiện cụm ùn tắc ban đầu tại từng thời điểm, và (ii) mở rộng vùng dựa trên cấu trúc mạng lưới và các tiêu chí không gian.

1.3.1 Phát hiện cụm ùn tắc theo snapshot

Tại mỗi thời điểm t , nhóm xác định tập các tuyến đường đang ở trạng thái ùn tắc dựa trên nhãn nhị phân $y_i(t) \in \{0, 1\}$. Đối với mỗi tuyến đường, tọa độ không gian được biểu diễn bằng điểm trung tâm của tuyến, được tính từ trung điểm giữa hai nút đầu và cuối của tuyến trong đồ thị giao thông.

Trên tập các tuyến đang ùn tắc tại thời điểm t , nhóm áp dụng thuật toán phân cụm DBSCAN để phát hiện các *cụm ùn tắc theo snapshot*. DBSCAN được lựa chọn do không yêu cầu số cụm cố định trước và có khả năng phát hiện các cụm với hình dạng bất kỳ, phù hợp với đặc điểm phân bố không gian của ùn tắc giao thông. Mỗi cụm thu được từ DBSCAN tại thời điểm t được xem như một *seed congestion cluster*, đóng vai trò khởi tạo cho quá trình xây dựng vùng giao thông động.

1.3.2 Chiến lược chọn seed vùng

Để đảm bảo tính cân bằng giữa các trạng thái giao thông khác nhau trong quá trình huấn luyện, nhóm áp dụng chiến lược chọn seed vùng kết hợp giữa các cụm ùn tắc và các vùng không ùn tắc. Cụ thể, trong quá trình sinh mẫu huấn luyện, 60% seed được lấy từ các cụm ùn tắc được phát hiện bởi DBSCAN tại thời điểm t , trong khi 40% seed còn lại được chọn từ các vùng không ùn tắc.

Các seed không ùn tắc có thể được tạo bằng cách áp dụng DBSCAN trên tập các tuyến không ùn tắc. Sau khi đã chọn ra các cụm kẹt hoặc không kẹt từ DBSCAN, mô hình sẽ lựa chọn ngẫu nhiên một vùng cụ thể với điều kiện tương ứng tại thời điểm t . Chiến lược này giúp mô hình học được cả các mẫu lan truyền ùn tắc lẫn các trạng thái giao thông ổn định, đồng thời giảm thiểu hiện tượng lệch lớp trong dữ liệu.

1.3.3 Mở rộng vùng theo cấu trúc mạng lưới

Từ mỗi seed congestion cluster, nhóm tiến hành mở rộng vùng giao thông để thu thập tập các tuyến ứng viên có khả năng liên quan đến động lực giao thông của vùng. Trước hết, tất cả các tuyến thuộc seed cluster được đưa vào tập vùng ban đầu. Sau đó, dựa trên cấu trúc đồ thị giao thông toàn cục, vùng được mở rộng bằng cách bổ sung các tuyến lân cận theo quan hệ topology, với số bước mở rộng (hop) được nhóm chọn là 1.

Quá trình mở rộng này nhằm đảm bảo rằng vùng giao thông không chỉ bao gồm các tuyến đang ùn tắc tại thời điểm hiện tại, mà còn bao phủ các tuyến có khả năng đóng vai trò trung gian hoặc chịu ảnh hưởng từ quá trình lan truyền ùn tắc trong tương lai.

1.3.4 Lọc vùng theo bán kính không gian

Để giới hạn phạm vi không gian của vùng giao thông và đảm bảo tính hợp lý về mặt vật lý, nhóm áp dụng một bước lọc cứng dựa trên bán kính không gian. Cụ thể, đối với mỗi seed cluster, một điểm trung tâm của vùng được xác định từ trung tâm hình học của các tuyến trong seed.

Với mỗi tuyến ứng viên thu được từ bước mở rộng theo topology, khoảng cách Haversine từ tuyến đó đến tâm seed được tính toán. Chỉ những tuyến có khoảng cách nhỏ hơn hoặc bằng ngưỡng $R_{\max}$ mới được giữ lại trong vùng giao thông cuối cùng. Ở mô hình này nhóm chọn bán kính là 1500m cho mỗi vùng. Bước lọc này giúp loại bỏ các tuyến ở quá xa seed cluster, vốn khó có mối liên hệ trực tiếp với sự kiện ùn tắc ban đầu.

1.3.5 Vùng giao thông chức năng

Sau các bước trên, vùng giao thông thu được không còn là một cụm không gian thuần túy, mà là một *vùng giao thông chức năng*, bao gồm các tuyến đường có mối liên hệ về không gian, cấu trúc mạng lưới và tiềm năng ảnh hưởng giao thông. Đáng chú ý, các tuyến nằm trong vùng giao

thông này không nhất thiết phải thuộc seed congestion cluster ban đầu, mà có thể nằm ngoài cụm nhưng vẫn có vai trò quan trọng trong động lực lan truyền ùn tắc.

Vùng giao thông chức năng này đóng vai trò là đơn vị đầu vào cho các bước tiếp theo, trong đó các mối tương quan giao thông có độ trễ sẽ được khai thác để xác định các liên kết chức năng và tập các tuyến mục tiêu cần dự đoán.

1.4 Khai thác tương quan giao thông và xác định tuyến mục tiêu

Sau khi xây dựng vùng giao thông chức năng như trình bày ở Mục ??, bước tiếp theo nhằm làm rõ các mối quan hệ ảnh hưởng giữa các tuyến đường trong vùng và xác định tập các tuyến mục tiêu cần dự đoán. Trong bối cảnh giao thông đô thị, ùn tắc không lan truyền một cách ngẫu nhiên mà thường tuân theo các mối quan hệ có hướng và có độ trễ theo thời gian, phản ánh động lực di chuyển của phương tiện trong mạng lưới giao thông. Do đó, việc khai thác các mối tương quan giao thông có xét đến độ trễ đóng vai trò then chốt trong việc xác định các tuyến có ảnh hưởng quan trọng.

1.4.1 Tương quan chéo có độ trễ trong vùng giao thông

Đối với mỗi vùng giao thông chức năng, nhóm chỉ xét các tuyến đường thuộc vùng này khi tính toán tương quan, nhằm đảm bảo tính cục bộ và giảm độ phức tạp tính toán. Với mỗi cặp tuyến đường (i, j) trong vùng, từ chuỗi thời gian vận tốc đã được chuẩn hóa $S_i(t)$ và $S_j(t)$, hàm tương quan chéo có độ trễ $X_{i,j}(\tau)$ được tính trên một cửa sổ thời gian lịch sử, với $\tau \in [-\tau_{\max}, \tau_{\max}]$.

Từ hàm tương quan chéo này, nhóm xác định hai đại lượng đặc trưng cho mối quan hệ giữa hai tuyến đường: (i) trọng số liên kết dương $W_{i,j}^{\text{pos}}$, phản ánh mức độ nổi trội của đỉnh tương quan so với nền nhiễu, và (ii) độ trễ tương ứng $\tau_{i,j}^{\text{pos}}$, xác định hướng và thời điểm ảnh hưởng mạnh nhất. Các đại lượng này được tính theo khuôn khổ mạng tương quan giao thông được đề xuất bởi Guo et al. [?].

Việc sử dụng $W_{i,j}^{\text{pos}}$ thay vì giá trị tương quan cực đại thuần túy cho phép loại bỏ các mối tương quan yếu hoặc ngẫu nhiên, đồng thời làm nổi bật các liên kết có ý nghĩa thống kê trong động lực giao thông. Trong khi đó, $\tau_{i,j}^{\text{pos}}$ cung cấp thông tin về quan hệ lead-lag giữa các tuyến, phản ánh hướng lan truyền tiềm năng của ùn tắc.

1.4.2 Lọc liên kết tương quan theo tiêu chí thống kê và không gian

Không phải mọi cặp tuyến có tương quan dương đều mang ý nghĩa chức năng trong bối cảnh lan truyền ùn tắc. Do đó, nhóm áp dụng một tập các tiêu chí lọc để chỉ giữ lại các liên kết tương quan mạnh và hợp lý về mặt không gian–thời gian. Cụ thể, một liên kết (i, j) được giữ lại nếu thỏa mãn đồng thời các điều kiện sau:

$$W_{i,j}^{\text{pos}} \geq W_{\min}, \quad D_{\min} \leq D_{i,j} \leq D_{\max}, \quad |\tau_{i,j}^{\text{pos}}| \leq \tau_{\text{cut}}, \quad (4)$$

trong đó $D_{i,j}$ là khoảng cách Haversine giữa hai tuyến đường i và j . Các ngưỡng $W_{\min}$, $D_{\min}$, $D_{\max}$ và τ_{cut} được lựa chọn nhằm cân bằng giữa việc giữ lại các liên kết có ý nghĩa và loại bỏ nhiễu.

Bước lọc này đảm bảo rằng các liên kết tương quan được sử dụng trong mô hình không chỉ mạnh về mặt thống kê, mà còn phù hợp với ràng buộc vật lý và động học của giao thông đô thị. Các liên kết có khoảng cách quá xa hoặc độ trễ quá lớn thường khó phản ánh mối quan hệ ảnh hưởng trực tiếp và do đó bị loại bỏ.

1.4.3 Xác định tập tuyến mục tiêu

Từ tập các liên kết tương quan thỏa mãn các tiêu chí lọc, nhóm xác định *tập các tuyến mục tiêu* trong mỗi vùng giao thông. Cụ thể, một tuyến đường j được xem là tuyến mục tiêu nếu nó

xuất hiện trong ít nhất một liên kết tương quan được giữ lại với bất kỳ tuyến nào khác trong vùng. Trong trường hợp số lượng tuyến mục tiêu quá lớn, nhóm chỉ giữ lại một tập con gồm các tuyến có trọng số liên kết W^{pos} lớn nhất.

1.5 Biểu diễn không gian bằng Laplacian Positional Encoding

Trong các mô hình Transformer tiêu chuẩn, cơ chế self-attention không tự thân nắm bắt được thông tin vị trí hay cấu trúc của các phần tử đầu vào, mà cần được bổ sung thông qua các cơ chế mã hóa vị trí (positional encoding). Đối với dữ liệu chuỗi, vị trí có thể được xác định một cách tự nhiên theo trật tự thời gian. Tuy nhiên, trong bài toán giao thông trên đồ thị, các tuyến đường không được sắp xếp theo một trật tự tuyến tính cố định, mà tồn tại trong một không gian phi Euclid được xác định bởi cấu trúc mạng lưới giao thông.

Do đó, để cung cấp thông tin không gian cho mô hình Transformer, nhóm sử dụng *Laplacian Positional Encoding*, một phương pháp mã hóa vị trí dựa trên phổ của đồ thị, đã được chứng minh là hiệu quả trong các mô hình Transformer cho dữ liệu đồ thị.

1.5.1 Xây dựng đồ thị không gian trong vùng giao thông

Đối với mỗi vùng giao thông chức năng được xây dựng ở các bước trước, nhóm xem vùng này như một đồ thị con $G_z = (V_z, E_z)$ của đồ thị giao thông toàn cục, trong đó mỗi đỉnh $v_i \in V_z$ tương ứng với một tuyến đường trong vùng, và các cạnh E_z được xác định dựa trên quan hệ kết nối topology giữa các tuyến trong mạng lưới giao thông.

Từ đồ thị con này, một ma trận kề $A_z \in \mathbb{R}^{|V_z| \times |V_z|}$ được xây dựng, trong đó $A_z(i, j) = 1$ nếu tồn tại cạnh nối giữa hai tuyến i và j trong đồ thị giao thông, và $A_z(i, j) = 0$ trong trường hợp ngược lại. Trên cơ sở đó, ma trận bậc D_z được xác định với $D_z(i, i) = \sum_j A_z(i, j)$.

1.5.2 Laplacian và phân rã phổ

Ma trận Laplacian của đồ thị vùng được định nghĩa dưới dạng Laplacian tổ hợp:

$$L_z = D_z - A_z. \quad (5)$$

Ma trận Laplacian này mã hóa đầy đủ cấu trúc kết nối của vùng giao thông và đóng vai trò như một toán tử khuếch tán trên đồ thị. Thực hiện phân rã phổ của L_z cho phép ta thu được hệ eigenvalue và eigenvector:

$$L_z = U_z \Lambda_z U_z^\top, \quad (6)$$

trong đó $U_z = [\mathbf{u}_1, \mathbf{u}_2, \dots, \mathbf{u}_{|V_z|}]$ là các eigenvector tương ứng với các eigenvalue được sắp xếp theo thứ tự không giảm $0 = \lambda_1 \leq \lambda_2 \leq \dots$.

Trong nghiên cứu này, nhóm chỉ sử dụng d_{spa} eigenvector ứng với các eigenvalue nhỏ nhất (bỏ qua eigenvector hằng ứng với $\lambda_1 = 0$ nếu cần), tạo thành ma trận mã hóa vị trí không gian:

$$\mathbf{P}_z = [\mathbf{u}_2, \mathbf{u}_3, \dots, \mathbf{u}_{d_{\text{spa}}+1}] \in \mathbb{R}^{|V_z| \times d_{\text{spa}}}. \quad (7)$$

Việc lựa chọn các eigenvector bậc thấp tương ứng với các thành phần tần số thấp của đồ thị cho phép mã hóa cấu trúc không gian một cách trơn (smooth) và mang tính toàn cục, phản ánh vai trò cấu trúc của mỗi tuyến đường trong vùng giao thông.

1.5.3 Ý nghĩa của eigenvector bậc thấp trong bối cảnh giao thông

Các eigenvector bậc thấp của Laplacian đồ thị có thể được xem như các “tọa độ cấu trúc” (structural coordinates) của các tuyến đường trong vùng. Các tuyến có vị trí cấu trúc gần nhau trong đồ thị (ví dụ cùng nằm ở trung tâm vùng hoặc cùng thuộc rìa vùng) sẽ có biểu diễn gần nhau trong không gian phổ, ngay cả khi chúng không kề nhau trực tiếp về mặt topology.

Trong bối cảnh giao thông đô thị, điều này đặc biệt quan trọng vì hiện tượng ùn tắc thường lan truyền một cách liên tục và có cấu trúc theo vùng, thay vì thay đổi đột ngột trên từng tuyến riêng lẻ. Việc sử dụng các eigenvector bậc thấp giúp mô hình nắm bắt được cấu trúc không gian tổng thể của vùng giao thông, đồng thời giảm độ nhạy với nhiễu cục bộ hoặc các liên kết không ổn định.

1.5.4 Tích hợp Laplacian Positional Encoding vào Transformer

Ma trận mã hóa vị trí không gian $\mathbf{P}_z$ được sử dụng như một *spatial positional encoding* cho các tuyến đường trong vùng giao thông. Đối với mỗi mẫu huấn luyện, mã hóa này được ghép với biểu diễn theo thời gian của dữ liệu đầu vào và được cộng vào embedding đầu vào của mô hình Transformer.

Bằng cách này, cơ chế self-attention trong Transformer không chỉ dựa trên sự tương đồng ngữ nghĩa của các đặc trưng giao thông theo thời gian, mà còn được điều kiện hóa bởi thông tin vị trí cấu trúc của các tuyến trong vùng. Điều này cho phép mô hình học được các tương tác không gian-thời gian phức tạp và vượt qua các giới hạn của các mô hình dựa trên cơ chế truyền thông tin cục bộ truyền thống.

Laplacian Positional Encoding do đó đóng vai trò then chốt trong việc cung cấp thông tin không gian cho mô hình Transformer encoder-only, tạo nền tảng cho việc học biểu diễn vùng giao thông một cách hiệu quả và có ý nghĩa về mặt cấu trúc.

1.6 Biểu diễn thời gian bằng Temporal Encoding

Bên cạnh cấu trúc không gian của mạng lưới giao thông, tín hiệu giao thông còn thể hiện tính chu kỳ mạnh theo thời gian, đặc biệt là theo *thời điểm trong ngày* (giờ cao điểm sáng/chiều) và *ngày trong tuần*. Do đó, để mô hình Transformer có thể phân biệt được các mẫu hình giao thông theo ngữ cảnh thời gian và khai thác được tính tuần hoàn, nhóm thiết kế một cơ chế *Temporal Encoding* để mã hóa thông tin thời gian và bổ sung vào embedding đầu vào.

1.6.1 Mã hóa vị trí tương đối theo bước thời gian

Đối với mỗi mẫu huấn luyện, dữ liệu đầu vào được cắt theo một cửa sổ thời gian dài L bước: $\{t - L + 1, \dots, t\}$. Vì Transformer không có khái niệm thứ tự nội tại giữa các token, nhóm sử dụng *relative position embedding* để mã hóa vị trí của mỗi bước thời gian trong cửa sổ này. Cụ thể, với chỉ số vị trí tương đối $p \in \{0, 1, \dots, L - 1\}$, mỗi vị trí được gán một vector embedding học được:

$$\mathbf{e}_p \in \mathbb{R}^{d_{\text{model}}}. \quad (8)$$

Cơ chế này giúp mô hình phân biệt rõ các quan sát gần hiện tại và các quan sát xa hơn trong quá khứ, từ đó học được động lực thay đổi của giao thông theo thời gian.

1.6.2 Mã hóa chu kỳ theo thời điểm trong ngày và ngày trong tuần

Để khai thác tính chu kỳ của giao thông, nhóm sử dụng các đặc trưng thời gian gồm: (i) thời điểm trong ngày đã chuẩn hóa $\text{tod}(t) \in [0, 1]$, và (ii) ngày trong tuần $\text{dow}(t) \in \{0, \dots, 6\}$. Các đặc trưng này được đưa về dạng biểu diễn lượng giác nhằm phản ánh tính tuần hoàn:

$$\mathbf{c}_{\text{tod}}(t) = [\sin(2\pi \text{tod}(t)), \cos(2\pi \text{tod}(t))], \quad (9)$$

$$\mathbf{c}_{\text{dow}}(t) = [\sin(2\pi \text{dow}(t)/7), \cos(2\pi \text{dow}(t)/7)]. \quad (10)$$

Sau đó, nhóm ghép các thành phần trên thành một vector 4 chiều và chiếu tuyến tính sang không gian d_{model} :

$$\mathbf{g}(t) = \mathbf{W}_t [\mathbf{c}_{\text{tod}}(t) \parallel \mathbf{c}_{\text{dow}}(t)] + \mathbf{b}_t, \quad \mathbf{W}_t \in \mathbb{R}^{d_{\text{model}} \times 4}. \quad (11)$$

1.6.3 Kết hợp Temporal Encoding vào embedding đầu vào

Đối với mỗi bước thời gian trong cửa sổ đầu vào, Temporal Encoding được tạo bằng cách cộng hai thành phần: (1) embedding vị trí tương đối theo chỉ số p và (2) embedding chu kỳ theo ngữ cảnh thời gian thực:

$$\mathbf{t}(p) = \mathbf{e}_p + \mathbf{g}(t - L + 1 + p), \quad p = 0, \dots, L - 1. \quad (12)$$

Vector $\mathbf{t}(p)$ sau đó được cộng vào embedding của tất cả các tuyến đường tại bước thời gian p . Nhờ đó, cơ chế self-attention có thể điều kiện hóa trên ngữ cảnh thời gian, giúp mô hình học được các mẫu hình giao thông mang tính lặp lại (ví dụ cao điểm sáng/chiều) và phân biệt các trạng thái giao thông giữa các ngày trong tuần.

Tóm lại, Temporal Encoding cung cấp cho mô hình Transformer một tín hiệu thời gian giàu ngữ cảnh, bổ sung cho Spatial Encoding dựa trên Laplacian, tạo nền tảng cho việc học biểu diễn không gian-thời gian trong phần kiến trúc mô hình trình bày ở mục tiếp theo.

2 Kiến trúc mô hình

2.1 Đầu vào của mô hình

Mô hình được thiết kế để xử lý dữ liệu giao thông theo vùng, trong đó mỗi mẫu đầu vào bao gồm chuỗi quan sát giao thông trong một cửa sổ thời gian, cùng với thông tin ngữ cảnh không gian và thời gian tương ứng. Cụ thể, đầu vào của mô hình bao gồm ba thành phần chính:

- **Chuỗi giao thông theo vùng:** lịch sử tốc độ giao thông của các tuyến đường trong cùng một vùng giao thông chức năng.
- **Thông tin thời gian:** bao gồm thời điểm trong ngày và ngày trong tuần, được mã hóa thông qua Temporal Encoding nhằm phản ánh tính chu kỳ của giao thông đô thị.
- **Thông tin không gian:** biểu diễn vị trí cấu trúc của các tuyến đường trong vùng thông qua Laplacian Positional Encoding dựa trên phổ đồ thị.

Ba thành phần này được kết hợp tại tầng embedding đầu vào và cung cấp cho Transformer encoder một biểu diễn thống nhất, trong đó mỗi token đồng thời mang thông tin giao thông, vị trí không gian và ngữ cảnh thời gian.

2.2 Attention score bias trong Transformer

Khác với Transformer tiêu chuẩn, trong đó trọng số attention chỉ phụ thuộc vào độ tương đồng giữa truy vấn (query) và khóa (key), mô hình của nhóm cho phép tích hợp thêm một *attention score bias* bằng cách cộng một thành phần điều chỉnh vào ma trận điểm attention trước khi áp dụng hàm softmax. Cụ thể, với một head attention, điểm attention giữa hai token i và j được tính như sau:

$$\alpha_{i,j} = \frac{\mathbf{q}_i^\top \mathbf{k}_j}{\sqrt{d_k}} + \gamma B_{i,j}, \quad (13)$$

trong đó $\mathbf{q}_i$ và $\mathbf{k}_j$ lần lượt là vector truy vấn và khóa tương ứng, d_k là kích thước không gian khóa, $B_{i,j}$ là attention bias phản ánh mối quan hệ chức năng giữa hai token, và γ là một hệ số vô hướng được học trong quá trình huấn luyện.

Sau đó, trọng số attention được chuẩn hóa theo softmax:

$$\text{Attn}_{i,j} = \frac{\exp(\alpha_{i,j})}{\sum_{j'} \exp(\alpha_{i,j'})}. \quad (14)$$

Attention bias $B_{i,j}$ được xây dựng bên ngoài mô hình Transformer và đóng vai trò mã hóa thông tin bổ sung về quan hệ giữa các tuyến đường trong cùng một vùng giao thông, chẳng hạn như mối tương quan giao thông có độ trễ hoặc mối liên hệ không gian–cấu trúc trong mạng lưới. Việc đưa bias vào dưới dạng một thành phần cộng cho phép mô hình điều chỉnh điểm attention mà không làm thay đổi cấu trúc cơ bản của cơ chế self-attention.

Khác với các Transformer đồ thị như Graphormer, trong đó bias cấu trúc thường được thiết kế cố định và phụ thuộc vào khoảng cách đồ thị toàn cục, attention bias trong mô hình của chúng tôi mang tính linh hoạt theo từng vùng giao thông và từng thời điểm. Hơn nữa, hệ số γ được học cho phép mô hình tự động cân bằng giữa thông tin cấu trúc được mã hóa trong bias và độ tương đồng ngữ nghĩa thuần túy giữa các token.

Trong quá trình thực nghiệm, nhóm đã từng làm mô hình không sử dụng attention score bias trong cùng một thiết lập huấn luyện. Kết quả cho thấy, mô hình không có attention bias kém hiệu quả hơn. Mô hình tích hợp attention bias giúp cải thiện độ ổn định của quá trình huấn luyện và mang lại hiệu năng dự báo tốt hơn so với Transformer tiêu chuẩn không có bias. Điều này cho thấy rằng việc đưa thông tin quan hệ chức năng trực tiếp vào điểm attention là một cơ chế hiệu quả để tăng cường khả năng mô hình hóa các tương tác không gian–thời gian trong bài toán giao thông đô thị.

2.3 Transformer encoder-only theo phong cách BERT

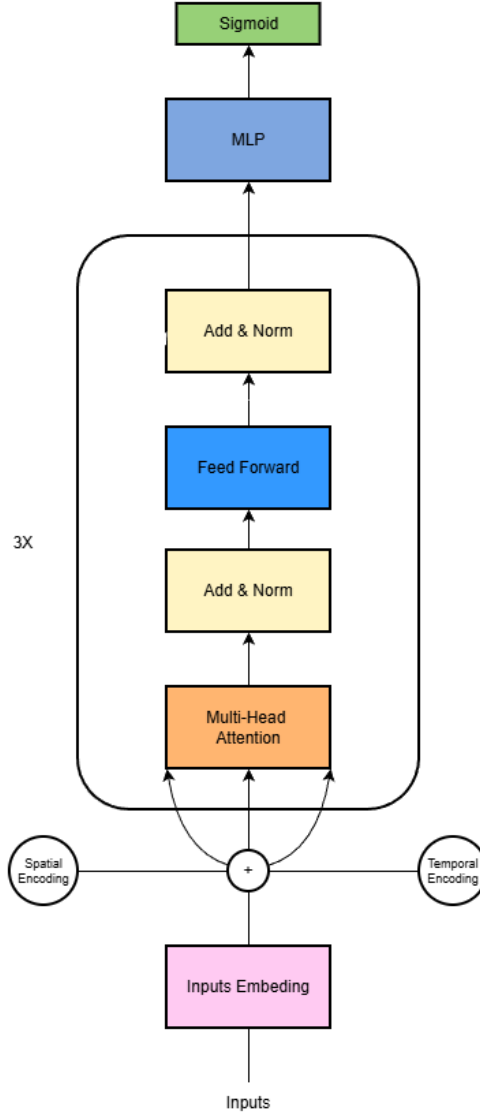


Figure 1: Kiến trúc mô hình Transformer encoder-only với Spatial và Temporal Encoding

Sau khi kết hợp các embedding đầu vào, dữ liệu được đưa vào một Transformer encoder-only theo phong cách BERT. Kiến trúc này bao gồm nhiều lớp encoder xếp chồng, mỗi lớp có cùng cấu trúc chuẩn gồm self-attention đa đầu và mạng feed-forward theo vị trí.

Mỗi lớp Transformer encoder trong mô hình bao gồm hai khối chính. Khối thứ nhất là *multi-head self-attention*, cho phép mỗi token trong vùng giao thông tương tác trực tiếp với tất cả các token khác trong cùng cửa sổ thời gian, nhờ đó mô hình có thể học được các quan hệ không gian-thời gian mang tính toàn cục.

Khối thứ hai là *feed-forward network* (FFN), bao gồm hai lớp tuyến tính với hàm kích hoạt phi tuyến ở giữa. FFN được áp dụng độc lập cho từng token và đóng vai trò tăng khả năng biểu diễn phi tuyến của mô hình sau bước attention.

Trong mỗi lớp encoder, chuẩn hóa lớp (Layer Normalization) được áp dụng trước mỗi khối self-attention và FFN, theo thiết kế *pre-norm*. Thiết kế này giúp ổn định quá trình huấn luyện, đặc biệt khi sử dụng nhiều lớp encoder. Các kết nối dư (residual connections) được sử dụng xuyên suốt để bảo toàn thông tin và hỗ trợ lan truyền gradient.

Để giảm hiện tượng overfitting, dropout được áp dụng tại các đầu ra của self-attention và bên trong mạng feed-forward. Việc sử dụng dropout ở nhiều vị trí giúp mô hình tổng quát tốt hơn khi huấn luyện trên dữ liệu giao thông có tính nhiễu và biến động cao.

2.4 Lớp dự đoán MLP

Sau khi đi qua các lớp Transformer encoder, mô hình sử dụng biểu diễn tại thời điểm cuối của cửa sổ thời gian để thực hiện dự đoán. Đối với mỗi tuyến đường trong vùng, embedding tương ứng được đưa qua một lớp MLP dùng chung tham số giữa các tuyến.

Lớp MLP này bao gồm một tầng ẩn với hàm kích hoạt phi tuyến, tiếp theo là dropout để giảm overfitting, và một tầng tuyến tính cuối cùng sinh ra một logit cho mỗi tuyến. Thiết kế MLP nông giúp giữ cho tầng dự đoán đơn giản, trong khi phần lớn năng lực mô hình được tập trung vào việc học biểu diễn không gian-thời gian thông qua Transformer encoder.

Kiến trúc dự đoán này phù hợp với giả định rằng các tuyến đường trong cùng một vùng giao thông chia sẻ các quy luật động học tương tự, đồng thời cho phép mô hình tổng quát hóa tốt sang các vùng giao thông khác nhau.

3 Kết quả thực nghiệm và đánh giá

3.1 Thiết lập bài toán và tiêu chí đánh giá

Trong phần này, nhóm trình bày kết quả thực nghiệm của mô hình đề xuất trên bài toán dự đoán xác suất ùn tắc sau 15 phút kể từ thời điểm quan sát cuối cùng của cửa sổ lịch sử. Các kết quả được báo cáo trên tập kiểm tra (test set), với mô hình được lựa chọn dựa trên giá trị F1-score cao nhất trên tập validation.

Do dữ liệu giao thông có tính chất chuỗi thời gian mạnh và tồn tại sự phụ thuộc theo thời gian, việc chia tập dữ liệu một cách ngẫu nhiên (random split) có thể dẫn đến hiện tượng *data leakage*, làm sai lệch kết quả đánh giá. Vì vậy, trong nghiên cứu này, nhóm áp dụng chiến lược chia tập theo thứ tự thời gian.

Cụ thể, toàn bộ dữ liệu được sắp xếp theo trục thời gian và được chia thành ba tập không giao nhau như sau:

- **Tập huấn luyện (train set):** chiếm 70% khoảng thời gian đầu tiên của dữ liệu, được sử dụng để học các tham số của mô hình.
- **Tập xác thực (validation set):** chiếm 15% khoảng thời gian tiếp theo, dùng để lựa chọn mô hình tốt nhất và điều chỉnh các quyết định huấn luyện (ví dụ lựa chọn epoch tốt nhất).
- **Tập kiểm tra (test set):** chiếm 15% khoảng thời gian cuối cùng, chỉ được sử dụng để đánh giá hiệu năng cuối cùng của mô hình.

Việc chia tập được thực hiện ở **cấp độ ngày (day-level split)**, tức là toàn bộ các mẫu thuộc cùng một ngày sẽ chỉ xuất hiện trong một tập duy nhất. Cách làm này giúp tránh việc các mẫu có mối liên hệ thời gian gần nhau bị phân tán giữa các tập khác nhau.

Ngoài ra, dữ liệu trong mỗi ngày được chia thành hai phiên giao thông độc lập:

- phiên buổi sáng (07:00–10:00),
- phiên buổi chiều (15:00–18:00).

Các tuyến đường mục tiêu dự đoán của mô hình: Dự đoán các tuyến có trong cluster được chọn sau khi DBSCAN và các tuyến liên quan mạnh tới các tuyến ở trong cluster đó.

Khi xây dựng các mẫu huấn luyện theo cửa sổ thời gian, nhóm không cho phép cửa sổ vượt qua ranh giới giữa hai phiên nhằm đảm bảo tính nhất quán về ngữ cảnh giao thông.

Đối với mỗi mẫu, nhãn dự đoán tại thời điểm $t + \Delta$ (với $\Delta = 15$ phút trong thí nghiệm này) được đảm bảo thuộc cùng tập dữ liệu với cửa sổ đầu vào. Điều này giúp loại bỏ hoàn toàn khả năng mô hình sử dụng thông tin tương lai trong quá trình huấn luyện hoặc đánh giá.

Ở mỗi batch (it trong ảnh), với tỉ lệ 60:40 cho kẹt và không kẹt, mô hình sẽ quyết định nên dùng dbscan cho kẹt hay không kẹt. Từ đó lấy ra các cluster tương ứng với trạng thái ùn tắc hiện tại, sau đó thì chọn ngẫu nhiên một cluster để train cho mô hình. Mỗi batch mô hình sẽ chọn ngẫu nhiên 8 clusters để train. Bên cạnh đó, tập valid, test cũng sẽ đổi clusters chứ không cố định sau mỗi epoch. Điều này nhằm kiểm tra và đảm bảo sự tổng quát hóa của mô hình và khả năng dự đoán của mô hình khi phải yêu cầu dự đoán một tuyến đường từ trạng thái một vùng bất kỳ

3.2 Quá trình huấn luyện và hội tụ

Mô hình được huấn luyện khoảng 150 epoch và ưu tiên chọn state có recall tốt nhất

```

===== Epoch 149/150 | threshold=0.3 =====
[E149][train] it=0000 loss=0.213812 eval_nodes=139 valid_nodes=163
[E149][train] it=0001 loss=0.276967 eval_nodes=537 valid_nodes=571
[E149][train] it=0002 loss=0.328699 eval_nodes=521 valid_nodes=557
[E149][train] it=0003 loss=0.213563 eval_nodes=93 valid_nodes=114
[E149][train] it=0004 loss=0.046399 eval_nodes=191 valid_nodes=232
[E149][train] it=0005 loss=0.074337 eval_nodes=148 valid_nodes=184
[E149][train] it=0006 loss=0.137197 eval_nodes=94 valid_nodes=103
[E149][train] it=0007 loss=0.221346 eval_nodes=111 valid_nodes=122
[E149][train] it=0008 loss=0.012122 eval_nodes=107 valid_nodes=199
[E149][train] it=0009 loss=0.287712 eval_nodes=62 valid_nodes=90
[E149][train] it=0010 loss=0.349959 eval_nodes=424 valid_nodes=492
[E149][train] it=0011 loss=0.088939 eval_nodes=102 valid_nodes=145
[E149][train] it=0012 loss=0.185938 eval_nodes=91 valid_nodes=165
[E149][train] it=0013 loss=0.177004 eval_nodes=156 valid_nodes=177
[E149][train] it=0014 loss=0.239131 eval_nodes=98 valid_nodes=113
[E149][train] it=0015 loss=0.267758 eval_nodes=513 valid_nodes=553
[E149][train] it=0016 loss=0.034158 eval_nodes=111 valid_nodes=174
[E149][train] it=0017 loss=0.355588 eval_nodes=536 valid_nodes=587
[E149][train] it=0018 loss=0.046529 eval_nodes=204 valid_nodes=239
[E149][train] it=0019 loss=0.107219 eval_nodes=118 valid_nodes=129
[E149][train] it=0020 loss=0.076917 eval_nodes=118 valid_nodes=160
[E149][train] it=0021 loss=0.066520 eval_nodes=122 valid_nodes=156
[E149][train] it=0022 loss=0.351513 eval_nodes=552 valid_nodes=572
[E149][train] it=0023 loss=0.094907 eval_nodes=134 valid_nodes=182
[E149][train] it=0024 loss=0.352265 eval_nodes=511 valid_nodes=534
[E149][train] it=0025 loss=0.195134 eval_nodes=134 valid_nodes=175
[E149][train] it=0026 loss=0.175848 eval_nodes=112 valid_nodes=113
[E149][train] it=0027 loss=0.102821 eval_nodes=155 valid_nodes=213
[E149][train] it=0028 loss=0.011300 eval_nodes=193 valid_nodes=315
[E149][train] it=0029 loss=0.362808 eval_nodes=63 valid_nodes=157
[E149][valid] it=0000 loss=0.092058 eval_nodes=146 valid_nodes=155
[E149][valid] it=0001 loss=0.133200 eval_nodes=125 valid_nodes=136
[E149][valid] it=0002 loss=0.073969 eval_nodes=176 valid_nodes=215
[E149][valid] it=0003 loss=0.118997 eval_nodes=134 valid_nodes=151
[E149][valid] it=0004 loss=0.077243 eval_nodes=97 valid_nodes=120
[E149][valid] it=0005 loss=0.328054 eval_nodes=531 valid_nodes=564
[E149][test] it=0000 loss=0.060800 eval_nodes=126 valid_nodes=118
[E149][test] it=0001 loss=0.147320 eval_nodes=167 valid_nodes=219
[E149][test] it=0002 loss=0.025482 eval_nodes=164 valid_nodes=179
[E149][test] it=0003 loss=0.086080 eval_nodes=181 valid_nodes=232
[E149][test] it=0004 loss=0.263022 eval_nodes=114 valid_nodes=127
[E149][test] it=0005 loss=0.387432 eval_nodes=466 valid_nodes=489
[E149][test] it=0006 loss=0.004080 eval_nodes=224 valid_nodes=257
[E149][test] it=0007 loss=0.095621 eval_nodes=135 valid_nodes=167
[E149] train: loss=0.181703 acc=0.9099 prec=0.8630 rec=0.9703 f1=0.9162 eval_nodes=6450 valid_nodes=7706
[E149] valid: loss=0.137103 acc=0.9247 prec=0.8683 rec=0.9709 f1=0.9167 eval_nodes=1209 valid_nodes=1341
[E149] test : loss=0.134640 acc=0.9296 prec=0.8586 rec=0.9730 f1=0.9123 eval_nodes=1577 valid_nodes=1798

```

Figure 2: Kết quả của Epoch 149

```

===== Epoch 150/150 | threshold=0.3 =====
[E150][train] it=0000 loss=0.084951 eval_nodes=141 valid_nodes=610
[E150][train] it=0001 loss=0.130708 eval_nodes=62 valid_nodes=138
[E150][train] it=0002 loss=0.156479 eval_nodes=68 valid_nodes=101
[E150][train] it=0003 loss=0.199562 eval_nodes=122 valid_nodes=181
[E150][train] it=0004 loss=0.038189 eval_nodes=175 valid_nodes=211
[E150][train] it=0005 loss=0.090633 eval_nodes=188 valid_nodes=270
[E150][train] it=0006 loss=0.218719 eval_nodes=118 valid_nodes=257
[E150][train] it=0007 loss=0.040474 eval_nodes=108 valid_nodes=165
[E150][train] it=0008 loss=0.169092 eval_nodes=146 valid_nodes=184
[E150][train] it=0009 loss=0.019628 eval_nodes=110 valid_nodes=154
[E150][train] it=0010 loss=0.285019 eval_nodes=87 valid_nodes=110
[E150][train] it=0011 loss=0.113105 eval_nodes=103 valid_nodes=115
[E150][train] it=0012 loss=0.310958 eval_nodes=471 valid_nodes=506
[E150][train] it=0013 loss=0.140859 eval_nodes=94 valid_nodes=143
[E150][train] it=0014 loss=0.265341 eval_nodes=113 valid_nodes=174
[E150][train] it=0015 loss=0.053681 eval_nodes=129 valid_nodes=150
[E150][train] it=0016 loss=0.100647 eval_nodes=122 valid_nodes=146
[E150][train] it=0017 loss=0.069963 eval_nodes=163 valid_nodes=211
[E150][train] it=0018 loss=0.047808 eval_nodes=127 valid_nodes=154
[E150][train] it=0019 loss=0.122624 eval_nodes=166 valid_nodes=200
[E150][train] it=0020 loss=0.123824 eval_nodes=189 valid_nodes=232
[E150][train] it=0021 loss=0.072426 eval_nodes=115 valid_nodes=160
[E150][train] it=0022 loss=0.208661 eval_nodes=489 valid_nodes=513
[E150][train] it=0023 loss=0.130642 eval_nodes=104 valid_nodes=167
[E150][train] it=0024 loss=0.397872 eval_nodes=505 valid_nodes=527
[E150][train] it=0025 loss=0.301261 eval_nodes=91 valid_nodes=121
[E150][train] it=0026 loss=0.046173 eval_nodes=81 valid_nodes=143
[E150][train] it=0027 loss=0.106107 eval_nodes=97 valid_nodes=184
[E150][train] it=0028 loss=0.077796 eval_nodes=121 valid_nodes=213
[E150][train] it=0029 loss=0.104851 eval_nodes=83 valid_nodes=195
[E150][valid] it=0000 loss=0.047367 eval_nodes=141 valid_nodes=159
[E150][valid] it=0001 loss=0.122780 eval_nodes=143 valid_nodes=173

```

```
[E150][valid] it=0001 loss=0.122780 eval_nodes=143 valid_nodes=173
[E150][valid] it=0002 loss=0.072419 eval_nodes=116 valid_nodes=121
[E150][valid] it=0003 loss=0.380467 eval_nodes=479 valid_nodes=586
[E150][valid] it=0004 loss=0.854957 eval_nodes=146 valid_nodes=162
[E150][valid] it=0005 loss=0.357570 eval_nodes=531 valid_nodes=566
[E150][test] it=0000 loss=0.062022 eval_nodes=123 valid_nodes=148
[E150][test] it=0001 loss=0.055352 eval_nodes=190 valid_nodes=223
[E150][test] it=0002 loss=0.003429 eval_nodes=122 valid_nodes=140
[E150][test] it=0003 loss=0.286319 eval_nodes=489 valid_nodes=503
[E150][test] it=0004 loss=0.038501 eval_nodes=162 valid_nodes=185
[E150][test] it=0005 loss=0.359523 eval_nodes=522 valid_nodes=537
[E150][test] it=0006 loss=0.141382 eval_nodes=134 valid_nodes=154
[E150][test] it=0007 loss=0.063432 eval_nodes=169 valid_nodes=233
[E150] train: loss=0.143335 acc=0.9298 prec=0.8738 rec=0.9637 f1=0.9166 eval_nodes=4688 valid_nodes=6635
[E150] valid: loss=0.159860 acc=0.9132 prec=0.8757 rec=0.9888 f1=0.9288 eval_nodes=1556 valid_nodes=1687
[E150] test : loss=0.136495 acc=0.9267 prec=0.8876 rec=0.9781 f1=0.9387 eval_nodes=1911 valid_nodes=2123

***** BEST checkpoint: epoch=108 valid_f1=0.9387 *****
[E108][test(best)] it=0000 loss=0.091626 eval_nodes=124 valid_nodes=142
[E108][test(best)] it=0001 loss=0.389924 eval_nodes=506 valid_nodes=548
[E108][test(best)] it=0002 loss=0.165267 eval_nodes=69 valid_nodes=84
[E108][test(best)] it=0003 loss=0.182842 eval_nodes=156 valid_nodes=182
[E108][test(best)] it=0004 loss=0.061043 eval_nodes=146 valid_nodes=159
[E108][test(best)] it=0005 loss=0.174387 eval_nodes=198 valid_nodes=134
[E108][test(best)] it=0006 loss=0.688295 eval_nodes=65 valid_nodes=75
[E108][test(best)] it=0007 loss=0.282460 eval_nodes=537 valid_nodes=583
[BEST] test ROC-AUC = 0.962996
{'best_epoch': 108, 'best_valid_score': 0.9386934673366834, 'best_metric': 'f1', 'threshold': 0.3, 'checkpoint': './outputs2/best_model.pt', 'out_dir': './outputs2', 'test_best': {'loss': 0.2544785946929753, 'metrics': {'acc': 0.8901227352425483, 'prec': 0.8497695852534562, 'rec': 0.9736008447729673, 'f1': 0.90748031496063}, 'counts': {'tp': 922, 'tn': 681, 'fp': 163, 'fn': 25}, 'eval_nodes': 1711, 'valid_nodes': 1987}, 'test_auc': 0.962996346418433, 'plots': {'loss_curves.png', 'metrics_train_valid_test.png', 'test_confusion_matrix.png', 'test_pr_curve.png', 'test_roc_auc_curve.png'}}
```

Figure 3: Kết quả của Epoch 150

Nhìn chung, ta có thể thấy loss giảm dần từ epoch 149 -> 150. Precision ở mức trên 85%, Recall ở mức trên 95% ổn định qua các epoch, trên tập train, test, valid. Eval nodes là tổng số node được dự đoán sau khi đã được chọn lọc từ tương quan mạnh nhất. Valid nodes là tổng số node trong cluster và eval nodes. Mỗi $eval_{nodes}$, $valid_{nodes}$, TP , FP , TN , FN trong epoch O tính như sau:

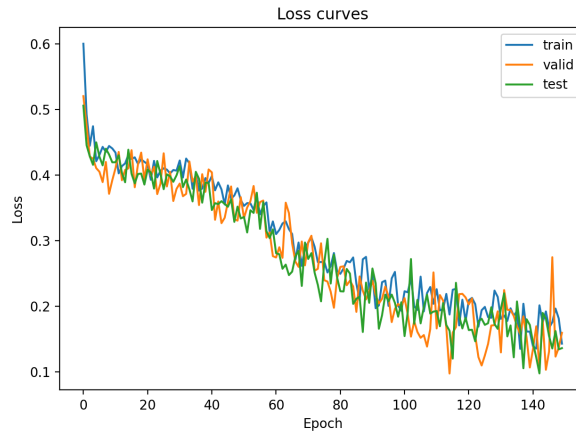


Figure 4: Hàm loss qua các epoch

Hình trên minh họa diễn biến hàm mất mát trên tập train, validation và test trong suốt quá trình huấn luyện. Có thể quan sát thấy rằng loss giảm dần và ổn định theo số epoch, cho thấy mô hình hội tụ tốt và không xuất hiện hiện tượng overfitting nghiêm trọng. Đường loss của ba tập có xu hướng bám sát nhau, phản ánh khả năng tổng quát hóa tốt của mô hình đối với dữ liệu chưa từng thấy.

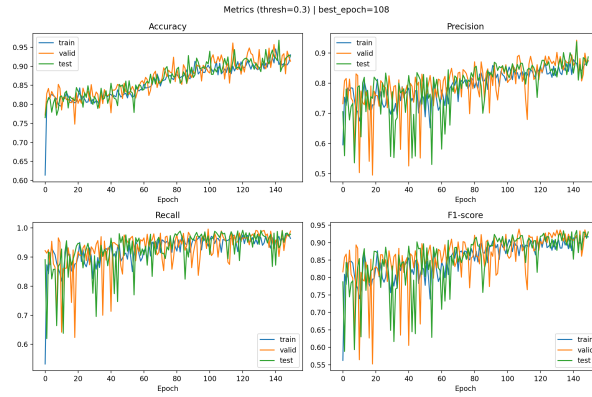


Figure 5: Accuracy, Precision, Recall, F1-score trên train, valid, test qua các epoch

Bên cạnh đó, có thể thấy rằng hình trên trình bày sự thay đổi của các chỉ số accuracy, precision, recall và F1-score theo epoch. Các chỉ số này tăng dần và đạt trạng thái ổn định sau một số epoch đầu, cho thấy mô hình dần học được các quy luật không gian–thời gian quan trọng của dữ liệu giao thông.

Dựa trên F1-score của tập validation, checkpoint tốt nhất được lựa chọn tại epoch 108, và mô hình tại epoch này được sử dụng để đánh giá cuối cùng trên tập test.

3.3 Đánh giá định lượng trên tập kiểm tra

Bảng dưới đây tổng hợp các chỉ số đánh giá chính trên tập test tại checkpoint tốt nhất. Mô hình đạt F1-score cao, cùng với recall lớn, cho thấy khả năng phát hiện các trạng thái ùn tắc trong tương lai một cách hiệu quả, trong khi vẫn duy trì mức precision ổn định.

Table 1: Kết quả định lượng trên tập test (dự đoán 15 phút sau).

Metric	Accuracy	Precision	Recall	F1-score
Test set	0.89	0.85	0.97	0.91

Giá trị recall cao cho thấy mô hình ít bỏ sót các trường hợp ùn tắc, điều này đặc biệt quan trọng trong các ứng dụng hỗ trợ điều hành giao thông, nơi việc phát hiện sớm ùn tắc có ý nghĩa thực tiễn lớn.

3.4 Phân tích đường cong PR và ROC

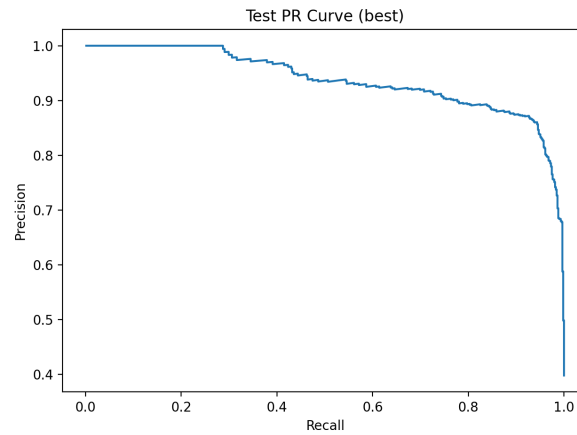


Figure 6: Precision-Recall curve

Hình trên trình bày đường cong Precision–Recall trên tập test tại checkpoint tốt nhất. Đường cong cho thấy mô hình duy trì precision cao trong một dải recall rộng, phản ánh khả năng cân bằng tốt giữa hai tiêu chí này ngay cả khi thay đổi ngưỡng phân loại.

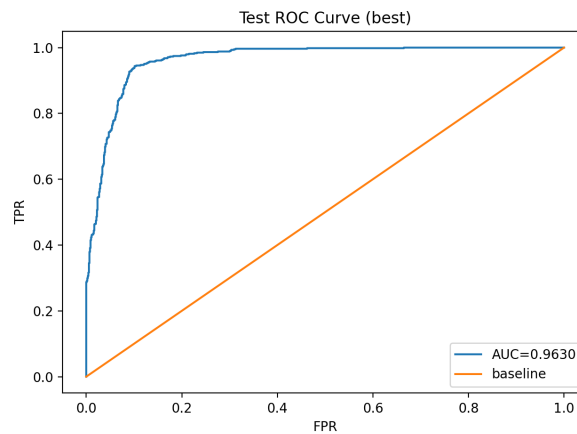


Figure 7: ROC,AUC

Hình trên minh họa đường cong ROC tương ứng, với diện tích dưới đường cong (AUC) đạt giá trị cao. Điều này cho thấy mô hình có khả năng phân biệt rõ ràng giữa hai lớp ùn tắc và không ùn tắc, độc lập với lựa chọn ngưỡng phân loại.

3.5 Phân tích ma trận nhầm lẫn

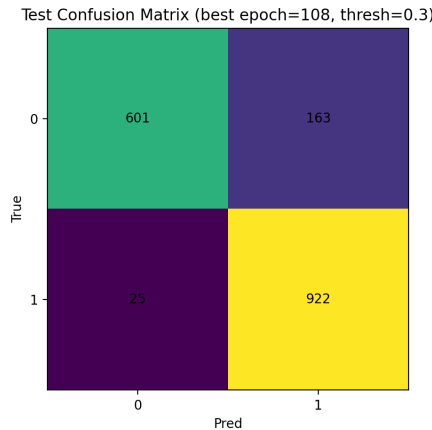


Figure 8: Ma trận nhầm lẫn

Để phân tích chi tiết hơn các dạng sai lệch của mô hình, nhóm sẽ xem xét ma trận nhầm lẫn trên tập test tại checkpoint tốt nhất. Có thể thấy số lượng false negative là tương đối nhỏ so với true positive, cho thấy mô hình ưu tiên phát hiện đúng các trạng thái ứn tắc trong tương lai. Ngoài ra, TP và TN khá cao cho thấy khả năng dự đoán tốt của mô hình. Bên cạnh đó, FP ở mức trung bình điều này làm giảm precision của mô hình.

3.6 Một số số liệu phản ánh khả năng học và dự đoán của mô hình sau 30 phút

3.7 Thảo luận

```
===== Epoch 149/150 | threshold=0.3 =====
[E149][train] it=0000 loss=0.314583 eval_nodes=534 valid_nodes=564
[E149][train] it=0001 loss=0.113628 eval_nodes=213 valid_nodes=250
[E149][train] it=0002 loss=0.264089 eval_nodes=538 valid_nodes=567
[E149][train] it=0003 loss=0.180254 eval_nodes=128 valid_nodes=161
[E149][train] it=0004 loss=0.099553 eval_nodes=145 valid_nodes=173
[E149][train] it=0005 loss=0.445242 eval_nodes=299 valid_nodes=368
[E149][train] it=0006 loss=0.234157 eval_nodes=99 valid_nodes=105
[E149][train] it=0007 loss=0.169155 eval_nodes=46 valid_nodes=68
[E149][train] it=0008 loss=0.053821 eval_nodes=153 valid_nodes=197
[E149][train] it=0009 loss=0.467809 eval_nodes=451 valid_nodes=522
[E149][train] it=0010 loss=0.267466 eval_nodes=581 valid_nodes=527
[E149][train] it=0011 loss=0.131540 eval_nodes=88 valid_nodes=162
[E149][train] it=0012 loss=0.199863 eval_nodes=113 valid_nodes=132
[E149][train] it=0013 loss=0.152353 eval_nodes=138 valid_nodes=202
[E149][train] it=0014 loss=0.283087 eval_nodes=72 valid_nodes=77
[E149][train] it=0015 loss=0.140877 eval_nodes=73 valid_nodes=90
[E149][train] it=0016 loss=0.429613 eval_nodes=526 valid_nodes=556
[E149][train] it=0017 loss=0.114446 eval_nodes=135 valid_nodes=158
[E149][train] it=0018 loss=0.111998 eval_nodes=181 valid_nodes=174
[E149][train] it=0019 loss=0.241471 eval_nodes=117 valid_nodes=222
[E149][train] it=0020 loss=0.053607 eval_nodes=217 valid_nodes=292
[E149][train] it=0021 loss=0.118487 eval_nodes=139 valid_nodes=200
[E149][train] it=0022 loss=0.203868 eval_nodes=86 valid_nodes=132
[E149][train] it=0023 loss=0.092999 eval_nodes=116 valid_nodes=171
[E149][train] it=0024 loss=0.146683 eval_nodes=134 valid_nodes=189
[E149][train] it=0025 loss=0.436714 eval_nodes=312 valid_nodes=356
[E149][valid] it=0000 loss=0.096423 eval_nodes=136 valid_nodes=157
[E149][valid] it=0001 loss=0.292676 eval_nodes=579 valid_nodes=623
[E149][valid] it=0002 loss=0.110351 eval_nodes=163 valid_nodes=181
[E149][valid] it=0003 loss=0.096632 eval_nodes=213 valid_nodes=258
[E149][valid] it=0004 loss=0.106866 eval_nodes=118 valid_nodes=154
[E149][valid] it=0005 loss=0.075934 eval_nodes=49 valid_nodes=68

[E149][test] it=0000 loss=0.049328 eval_nodes=183 valid_nodes=213
[E149][test] it=0001 loss=0.081759 eval_nodes=202 valid_nodes=239
[E149][test] it=0002 loss=0.063241 eval_nodes=108 valid_nodes=125
[E149][test] it=0003 loss=0.089119 eval_nodes=158 valid_nodes=173
[E149][test] it=0004 loss=0.388287 eval_nodes=375 valid_nodes=430
[E149][test] it=0005 loss=0.121637 eval_nodes=96 valid_nodes=136
[E149][test] it=0006 loss=0.094594 eval_nodes=132 valid_nodes=155
[E149] train: loss=0.210280 acc=0.8882 prec=0.8323 rec=0.9531 f1=0.8886 eval_nodes=5466 valid_nodes=6615
[E149] valid: loss=0.129814 acc=0.9205 prec=0.8532 rec=0.9812 f1=0.9127 eval_nodes=1258 valid_nodes=1441
[E149] test : loss=0.126852 acc=0.9115 prec=0.7943 rec=0.9088 f1=0.8477 eval_nodes=1254 valid_nodes=1471
```

Figure 9: Epoch 149

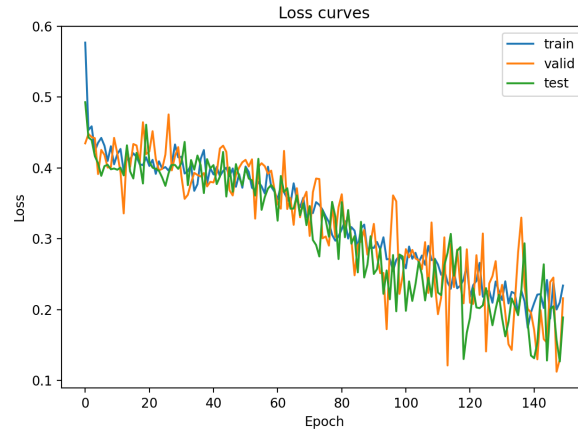


Figure 10: Hàm loss qua các epoch

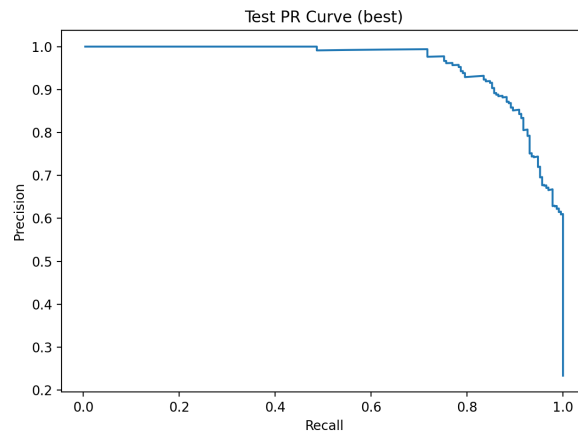


Figure 11: Precision-Recall curve

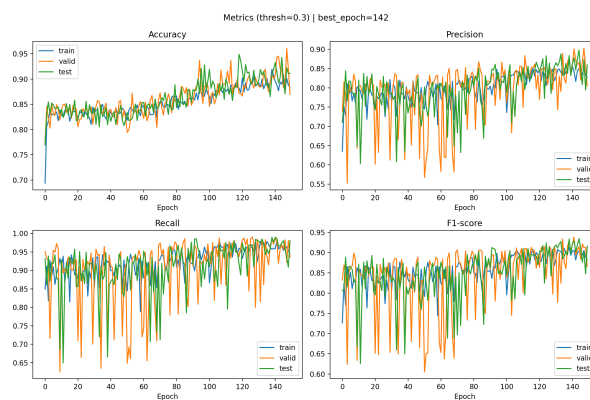


Figure 12: Precision-Recall curve

Ở đây mô hình được nạp và học dữ liệu lại và điều chỉnh để cho đúng với nhãn kẹt hoặc không kẹt 30 phút sau. Nhìn chung mô hình vẫn học được tốt, các hàm loss vẫn giảm đều và số liệu vẫn cho thấy mô hình có khả năng học được tốt

Kết quả thực nghiệm cho thấy mô hình Transformer đề xuất, với việc tích hợp bias vào attention score và các mã hóa không gian-thời gian, có khả năng dự báo hiệu quả trạng thái ứn

tắc trong tương lai ngắn hạn (15 phút, 30 phút, ...). Sự ổn định của quá trình huấn luyện và các chỉ số đánh giá cao trên tập test cho thấy mô hình không chỉ học tốt trên dữ liệu huấn luyện mà còn tổng quát hóa tốt sang dữ liệu chưa từng thấy.